PrepIQ — Company & Role-Aware Interview Prep Platform

Build For Bharat 2.0 — PRD & TRD (working title, rename freely)

Sep 27, 2026 · @Baller

Executive Summary

PrepIQ is a company- and role-aware interview preparation platform that turns real, publicly available interview experiences into a personalized, prioritized revision plan for two groups: students preparing for campus placements and working professionals preparing to switch to a new (often larger) company.

Instead of generic prep content, PrepIQ analyzes historical interview data per company, role, and experience level to compute a topic-weightage profile (what's actually asked, and how often), then compares it against the user's self-assessed or quiz-tested skill level to generate a ranked revision plan, plus a spaced-repetition layer that resurfaces topics the user is likely to have forgotten.

This directly targets the hackathon's Skill Gap Analysis, Career Intelligence, and Talent Development focus areas within the Intelligent Talent and Workforce Ecosystem theme, going beyond descriptive dashboards by using an LLM-driven extraction and scoring pipeline to produce actionable, individualized output.

Problem Statement & Target Stakeholders

Core pain points

1. **Unfocused revision** — candidates don't know what to prioritize; they revise broadly instead of what a specific company/role actually tests.
2. **Forgotten fundamentals** — topics learned months or years earlier fade without a structured revision/reminder system.
3. **No company-specific visibility** — every company, and every role within it, has a different interview pattern, round structure, and topic emphasis, but this information is scattered across forums, blog posts, and word of mouth.

Target stakeholders

Persona	Context	Primary need
Campus placement student	Fresher, DSA-heavy rounds: OA + technical + HR	What to revise, in what order, for this specific company
Working professional (job switcher)	2+ years experience, targeting a lateral move to a larger company	Refresh rusty fundamentals plus prep for system design and experience-based rounds, all in one place

Both personas share the same underlying need — a data-backed, personalized prep plan — but differ in round emphasis (heavier DSA for freshers, heavier system design and behavioral rounds for experienced hires). The product handles this with an `experience_level` parameter rather than building a separate product for each persona.

PRD — Goals & User Stories

Product goals

1. Give any user a data-backed, ranked list of what to study for a specific company, role, and experience level within seconds of choosing them.
2. Replace static "company-wise interview experience" reading with a structured, queryable topic-weightage model.
3. Reduce forgotten-fundamentals risk with a lightweight revision tracker that resurfaces decaying topics before an interview date.
4. Demonstrate a real analytical/AI pipeline (LLM-based extraction plus gap scoring), not a static lookup table, per the hackathon's requirement to go beyond visualization.

Success metrics (for the hackathon demo and beyond)

- Time-to-first-insight: under 10 seconds from selecting company, role, and experience to seeing a ranked revision plan.
- Coverage: structured topic-weightage data for at least 15–20 seeded companies across both fresher and experienced tracks.
- Personalization signal: the gap-priority ranking visibly changes when a user's self-assessed skill level changes.
- Judge-facing: a live, working demo end to end (not slides) covering Problem → Data → Insight → Solution.

Key user stories

- As a final-year student, I want to enter the company and role I'm interviewing for so that I can see exactly which topics and round types to expect.
- As a student, I want a prioritized revision list based on my current skill level so that I don't re-study what I already know well.
- As a working professional, I want to select my experience level so that my plan weighs system design and behavioral prep appropriately instead of pure DSA grinding.
- As a returning user, I want the app to remind me of topics I marked as learned a while ago so that I don't walk into the interview having forgotten them.
- As any user, I want to see that the round structure and topic weightage are derived from real interview data, not opinion, so that I trust the plan.

PRD — Feature Scope

In scope for the hackathon MVP

Feature	Description
Company + role + experience selector	User picks from a seeded list (about 15–20 companies x 2–3 roles x fresher/experienced)
Topic-weightage view	Data-backed breakdown of round structure and topic importance for that selection
Skill self-check	Short quiz or self-rating per topic to establish the user's current level
Personalized gap-priority list	Ranked revision list: topic importance x skill gap
Revision tracker (decay-based reminders)	Marks topics as "learned on [date]" and resurfaces them as they age past a set threshold
Data pipeline (offline)	LLM-based extraction of structured topic/round data from curated interview experience text

Out of scope for the hackathon (future roadmap)

- Live scraping/crawling of GeeksforGeeks, LeetCode Discuss, Glassdoor, etc. (a curated static dataset is used instead, for demo reliability)

- Full spaced-repetition scheduling (SM-2 style) — MVP uses a simpler linear decay heuristic
- Resume parsing / automatic skill detection
- Mock interview simulation (a natural v2 feature; not required to prove the core thesis)
- Multi-language support
- Native mobile app (a responsive web app is enough for the demo)

TRD — Functional & Non-Functional Requirements

Functional requirements

1. FR1 — The system shall let a user select a company, a role, and an experience level (fresher / 1–3 YOE / 4+ YOE) from a predefined list.
2. FR2 — The system shall return a topic-weightage profile (topics, round structure, relative importance) for that selection, computed from structured interview-experience data.
3. FR3 — The system shall let a user self-rate or quiz-check their current proficiency on each topic in the weightage profile.
4. FR4 — The system shall compute a gap-priority score per topic as a function of topic importance for the selection and (1 minus user proficiency), and return a ranked revision list.
5. FR5 — The system shall let a user mark a topic as "reviewed" with a timestamp, and shall recompute a decay-adjusted "needs revision" flag as time passes.
6. FR6 — The offline data pipeline shall ingest raw interview-experience text and, via an LLM extraction step, output structured records (company, role, experience level, topics, round type, recency).
7. FR7 — The system shall persist all of the above per user account so a plan can be revisited across sessions.

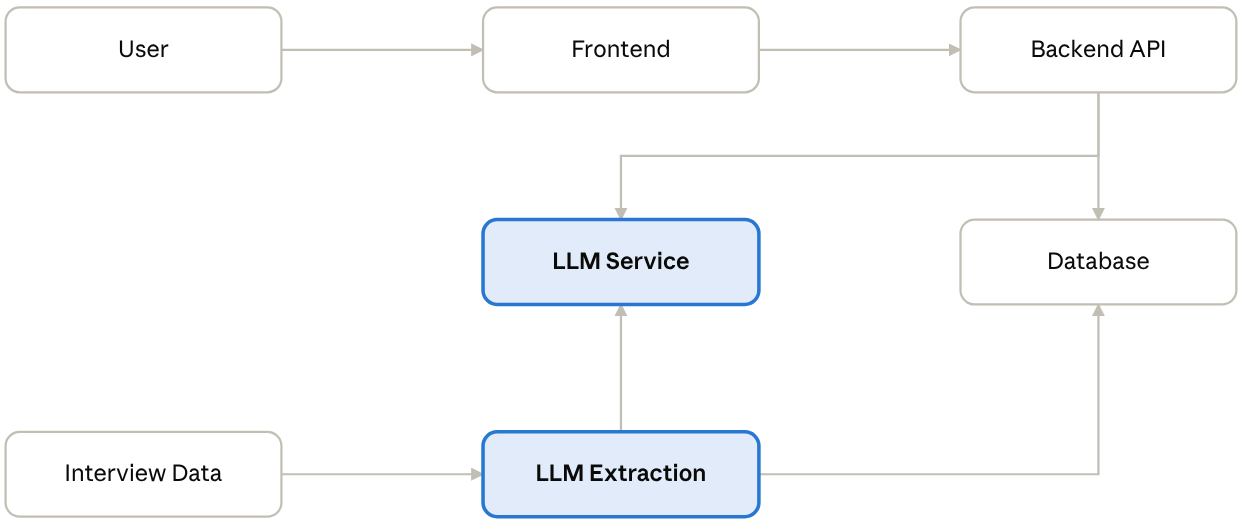
Non-functional requirements

Category	Requirement
Performance	Weightage and gap-priority computation returns in under 2 seconds for a seeded company/role pair
Scalability	Data model supports adding a new company/role without code changes: pure data insertion

Category	Requirement
Reliability	The demo path (company selection to ranked plan) works fully offline from cached/seeded data, independent of live scraping or third-party site uptime
Data integrity	Every extracted topic/round claim is traceable back to its source interview-experience record
Security	User accounts use hashed credentials; no plaintext resumes or unnecessary personal data stored
Cost control	LLM extraction runs as a one-time offline batch job per company, not per user request, to keep inference cost bounded

System Architecture

A live request path and an offline extraction pipeline both feed the database



architecture · live request path + offline extraction pipeline

Requests flow from the user through the frontend to the backend API, which reads and writes the database directly and calls the LLM service for on-demand features such as topic explanations. Separately, an offline pipeline reads curated interview-experience

text, calls the same LLM service to extract structured topics and round data, and writes the result into the database, so the live app never waits on an LLM call for its core ranked-plan output.

Tech Stack

Layer	Technology	Why
Frontend	React + Vite, Tailwind CSS	Fast to build, matches an existing web dev skillset, easy to make the ranked-plan UI feel responsive
Backend API	Node.js + Express (or FastAPI if the team prefers Python for the data pipeline)	Familiar, quick to stand up REST endpoints under hackathon time pressure
Database	PostgreSQL (or SQLite for a lighter demo)	Relational fit for company/role/topic/user-progress tables; easy local setup
AI / LLM	Claude API (or OpenAI API)	Used offline for structured extraction from raw interview text, and optionally for short topic explanations
Auth	Simple JWT-based auth or Firebase Auth	Low setup overhead, good enough for a hackathon demo login
Hosting (frontend)	Vercel or Netlify	One-command deploy, free tier, works well with React
Hosting (backend)	Render or Railway	Free or low cost, quick to deploy an Express/FastAPI service
Data curation	Python scripts plus a spreadsheet/JSON seed file	Fastest path to a reliable 15–20-company dataset without fighting live scraping during the hackathon

This stack leans on tools already familiar from web dev and DSA work, so hackathon hours go to the analytical pipeline and UX rather than learning new infrastructure.

Backend Details

Core data models

Table	Key fields
`companies`	id, name, industry
`roles`	id, company_id, title, experience_level (fresher / mid / senior)
`topics`	id, name, category (DSA, system design, CS fundamentals, behavioral)
`role_topic_weightage`	role_id, topic_id, weight (0–1), round_type, source_count
`interview_experiences`	id, company_id, role_id, raw_text, source_url, date_posted
`users`	id, email, password_hash, created_at
`user_skill_ratings`	user_id, topic_id, self_rating (0–5), last_reviewed_at
`revision_plans`	user_id, role_id, generated_at, plan_json

Core algorithm: gap-priority scoring

For a chosen role (company + role + experience level):

```
P_t = w_t \times (1 - s_t)
```

`P_t` is the priority score for topic *t*, `w_t` is that topic's weightage for the selected role (from `role_topic_weightage`, normalized 0 to 1), and `s_t` is the user's normalized skill rating for that topic (0 to 1, from self-rating or quiz score). Topics sort descending by `P_t` to produce the ranked revision list. A simple linear decay adjusts `s_t` downward the longer it has been since `last_reviewed_at`, which is what resurfaces topics the user is likely to have forgotten.

Key REST API endpoints

Method & Path	Purpose
`GET /companies`	List seeded companies
`GET /companies/:id/roles`	List roles and experience levels for a company
`GET /roles/:id/weightage`	Return the topic-weightage profile for a role
`POST /users/:id/skills`	Submit self-ratings or quiz results for topics
`GET /users/:id/plan?role_id=`	Return the ranked, personalized revision plan
`POST /users/:id/topics/:topicId/review`	Mark a topic reviewed, resetting its decay timer
`POST /admin/extract`	Trigger the offline LLM extraction job for a company (admin/dev use)

Data Acquisition & Processing Pipeline

Candidate public data sources

Source	What it provides	Notes
GeeksforGeeks company-wise interview experiences	Round-by-round write-ups per company	Large archive; good starting point for curation
LeetCode Discuss (company-tagged posts)	Community-shared interview questions and DSA topics	Useful for topic frequency signals
AmbitionBox / Glassdoor interview reviews	Round structure, difficulty, some behavioral context	Good for HR/behavioral round detail
Manually curated notes (own and peers' interview experiences)	High-trust, first-hand data	Small volume but very reliable for the seeded set

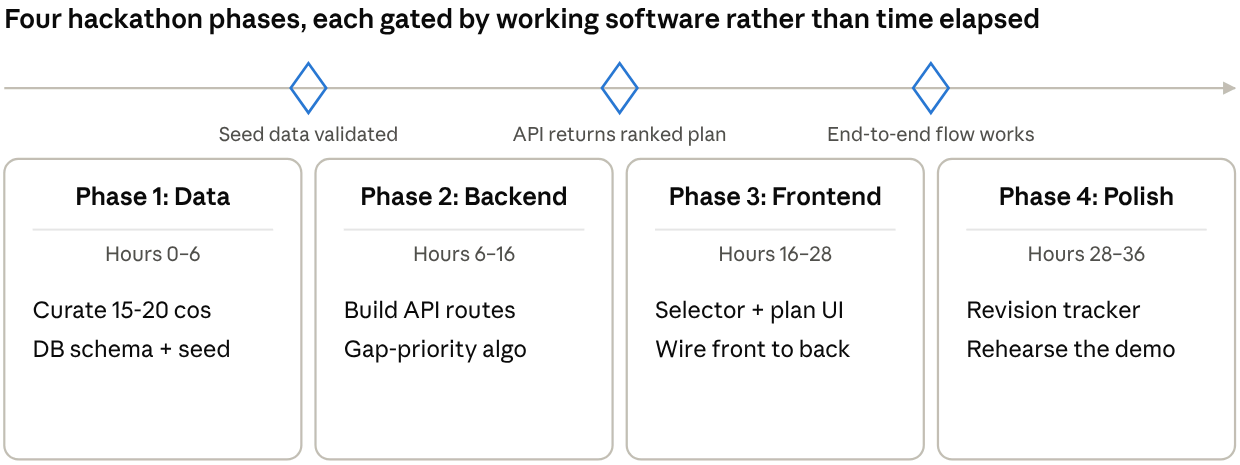
Curation approach for the hackathon

Rather than live scraping, which is fragile, rate-limited, and a demo-day risk, the plan is to manually collect and lightly clean raw text for about 15–20 companies across fresher and experienced tracks, and store it as a static seed dataset before the pipeline runs.

Processing pipeline (offline, run once per company/role, not per user request)

1. **Ingest** — load raw interview-experience text per company/role into `interview_experiences`.
2. **Extract** — send each record to the LLM with a structured-extraction prompt asking for topics mentioned, round type, relative emphasis, and recency; parse the response into structured fields.
3. **Aggregate** — combine extractions across all records for a role into a normalized `role_topic_weightage` table (weight = frequency-weighted, recency-weighted).
4. **Validate** — spot-check aggregated weightage against source text for a handful of roles to catch extraction errors before the demo.
5. **Serve** — the backend reads only the aggregated table at request time; raw text and the LLM are never on the live user-request path.

Hackathon Build Plan



build plan · 4 phases, 3 gates

This phasing maps directly onto what the problem statement asks for: Phase 1 covers Problem Identification and Data Acquisition; Phase 2 covers Data Preparation and the core Analytical Approach; Phase 3 turns that into a working Solution the judges can click through; Phase 4 sharpens the Insights/Predictions story and rehearses how the Real-

World Impact gets presented. Each gate is a working-software checkpoint, not just a clock check: if a gate isn't met on time, cut scope from the next phase's "nice to have" items rather than skipping validation.

Risks & Mitigations

Risk	Impact	Mitigation
Live scraping breaks or gets rate-limited during the hackathon	Demo fails or shows stale/broken data	Use a pre-curated static seed dataset (15–20 companies) instead of live scraping for the demo
LLM extraction produces noisy or inconsistent topic labels	Weightage profile looks unreliable to judges	Spot-check extracted output against source text for the seeded companies; keep the prompt tightly structured with a fixed topic taxonomy
Scope creep (trying to cover too many companies/roles)	Nothing works end to end by the deadline	Hard-cap the seed set at about 15–20 companies x 2–3 roles; treat anything beyond that as roadmap, not MVP
Judges question whether this is "just a lookup table"	Weakens the technical story	Show the gap-priority formula live, and show the plan changing when self-rated skill changes
Personalization feels shallow with only self-rated skill and no real assessment	Gap scores feel arbitrary	Add a short, topic-tagged quiz (even 1–2 questions per topic) instead of relying on self-rating alone
Correlation vs. causation concerns in weightage claims	Overclaiming what the data shows	Frame weightage as "how often this came up in real interview reports," not a guarantee of what will be asked

Risk	Impact	Mitigation
Time runs out before the revision-tracker/decay feature is built	A stated feature is missing from the demo	It is explicitly the last phase in the build plan; cut it first if time is short, since the core weightage and gap-scoring flow is the thesis